

Geographic Location Predictor

Raghavan Rajkumar, Owen Nguyen, Madison Shaw, Jack Trask, Bryce Kassiotis

1. Problem and Motivation

This project is about making a model which classifies images of outdoor settings by their geographic location. The main use case of this model would be to use for games such as GeoGuesser, where a player is supposed to guess where they are on a world map based off images of their surroundings. This is challenging, as it requires our model to be able to distinguish similar images from different parts of the world by analyzing the surroundings. Furthermore, many clues for geographic location vary greatly between different images. The model must therefore be able to combine many different subtle features rather than just focusing on a few.

Deep learning is appropriate as CNNs can automatically learn hierarchical visual features from images. Rather than finding and manually specifying which geographic features to use, a CNN can learn that during training. Our project will initially frame itself as a classification based off different regions of the world, with the same number of images to be trained per region.

Some research questions which can be posed include how well can the model predict a geographic location based only on visual data? How well does the pretrained model work when it is presented with new real-world data? What types of locations or environments confuse the model and cause false classifications?

2. Proposed Solution and Course Connection

Our team proposes using a CNN pre-trained on ImageNet or ResNet-50, which are large databases used for visual object recognition software. Pre-trained models allow models to leverage features from millions of labeled images, which reduces training time and improves generalization (Wi et al., 2026). Softmax regression will be used to convert the raw output scores from the classification head into a probability distribution over the defined geo-cells, which allows the model to predict the most likely location for the input image.

The model will be given an RGB image of a landscape or street view that is resized and cropped to the model's expected size (px). The output will be the predicted region of the location where the image was taken.

Classification loss (L_{cls}) and regression loss ($L_{haversine}$) will be used together to calculate loss.

$$L = L_{cls} + \lambda \cdot L_{haversine}$$

The classification loss is the cross-entropy loss over our geo-cells and will penalize the model for putting a low probability on the correct geo-cell. The regression loss uses the haversine distance function to find the great-circle distance between the predicted latitude/longitude and the ground truth latitude/longitude (Haas et al., 2023). Lambda will weigh how much the regression loss matters relative to classification loss. A large lambda means that regression dominates the loss function, while a smaller lambda means that regression loss will fine-tune.

The Adam optimizer will be used due to its faster convergence on deep neural networks. Parameters with noisy or infrequent gradients will get more conservative updates, while parameters with consistent gradients will change with larger and more decisive steps. For training, the pre-trained model will be frozen, and we will add a smaller head that maps our training image features to location classes. After the head is trained, we will unfreeze the body and fine-tune the model using a much smaller learning rate. By training the model using this technique, we can avoid overfitting on our smaller dataset, and the model comes with a strong starting point of basic feature detection.

Depending on the size of our dataset, we will introduce data augmentation hyperparameters, such as cropping, horizontal flipping, and color jitter. This allows the model to see slightly different versions of the same image and scene and is pushed towards the more meaningful and relevant features. We also plan to track model performance during training using a validation set. If the validation performance stops improving after epochs, we will roll back to the best checkpoint, which avoids overfitting.

Course concepts that we will use in our project include convolutional neural networks, softmax regression, cross-entropy loss, Adam, and early stopping.

3. Data Plan

The dataset used will be OpenStreetView-5M (OSV-5M) [Astruc et al., CVPR 2024]. This includes public crowdsourced street images from Mapillary, with over 5.1 million images covering 225 countries and territories. The dataset in full is 259 GB; we will download a subset of the training image data, along with the full official test set, which is downloadable on its own.

Inputs are single RGB street-level images, each labeled with GPS latitude and longitude. Coordinates are converted into classification targets by splitting the world into many cells with roughly equal image counts. The metadata included such as country and climate zones can be used for error analysis. Images will be resized to 224 by 224, normalized using ImageNet statistics, and training will be done using randomized cropping and slight color adjustments.

We will split the data 90/10 between training and validation, using the official test set. The images will be grouped by location to avoid very similar frames showing up in the training set. The test set will only be used at the end, after final model selection to get final accuracy values.

With the concentration of images in Europe and North America, geographic imbalance is the main risk of this dataset. The geographic cells will be used to offset this effect as much as possible. Faces and license plates are blurred in the data, and it will be credited as the licensing requires.

4. Model Evaluation and Analysis

We will exclusively evaluate our model's performance on OSV-5M's test split. This is so the reported performance will be based on unseen locations.

Primary Metrics

Metric	What It Measures	Why
Median and Mean Haversine distance error (km)	Straight-line distance between predicted coordinates (coordinates of predicted geocell) and true coordinates	Continuous error is necessary because simply using classification accuracy would harshly penalize a location one geocell away, even though that should be penalized much less. Also, we are using median and mean haversine distance because we don't want a few awful misses to make the model seem bad, or a good median to hide the fact that some predictions are very wrong.
Accuracy at 1, 25, 200, 750, 2,500 km	For each threshold distance, what percentage of test images had a predicted location within that many km of that location	Makes distance error more human-interpretable.

Accuracy for country, region, sub-region, city	Classification accuracy against OSV-5M's country, region, subregion, city labels	Uses OSV-5M's ground-truth labels.
GeoScore	$\text{GeoScore} = 5000 \cdot e^{\left(-\frac{\text{dist}}{1492.7}\right)}$ Where dist is the straight-line distance error.	Close guesses are rewarded generously (small distances barely reduce score), but the penalty grows fast as distance increases. Best score: 5000. Worst score: 0.

Model and Evaluation Targets

Our model will classify over OSV-5M's quadtree geocells at three different granularities: quadtree_10_500, quadtree_10_2500, and quadtree_10_25000. This means we will train and evaluate the model over the three separate quadtree geocells to determine the tradeoff between classification accuracy and location precision as geocell size grows/shrinks.

Success Criteria

Success will be measured against the two baselines on our test split (random geocell and most-common country) our model should perform significantly better against both of these. However, since we aren't using all 4.8M images from OSV-5M, we cannot expect it to perform better than OSV-5M's published baseline of GeoScore 3361, mean error 1,814 km, and country/region/sub-region/city accuracy of 68%/39.4%/10.3%/5.9%.

Analysis Plan

- Granularity Ablation: Compare all three granularities to determine classification accuracy and location precision tradeoffs
- Per-continent breakdown: Determine accuracy and error per continent
- Learning curves: Validation GeoScore vs training size (50k, 150k, 300k, etc.)
- Failure analysis: Grad-CAM analysis on cases with bad error

Expected Tables/Figures

- Main results table
 - Rows: Each granularity

- Columns: Median and Mean Haversine distance error (km), Accuracy at 1 / 25 / 200 / 750 / 2,500 km, Accuracy for country region / sub-region / city, Geoscore
- Learning curve plot
 - X-axis: Training set size
 - Y-axis: Average validation GeoScore
- Per-continent bar chart
 - X-axis: Continents
 - Y-axis: Average validation GeoScore
- 4-6 Grad-CAM examples

5. Feasibility, Resources, and Risks

There are a few risks that we will consider when implementing this project. The download and preprocessing for this project will be big, so we should plan to pull a subset early and resize the images once. Fine-grained accuracy will also be low, so we will focus more on accuracy within general regions and countries rather than more specific places on the map. Due to the unequal allocation of data available among all countries and regions, we will limit the scope of the project to fifty countries with the most data available. A standard modern GPU should be feasible for this project, and we will be using Python, PyTorch, and torchvision for the project framework. For the dataset, we will download a subset of OSV-5M from Hugging Face.

6. Team Responsibilities

Owen Nguyen: Download subset and test set, resize images to 224x224, build geocells (250, 1000, and 4000 versions), create location grouped train and validation split

Bryce Kassiotis: Build PyTorch training pipeline, use ResNet50 to fine tune and other methods, AdamW, checkpointing. Run first full training

Madison Shaw: Loss and prediction heads: Implement distance aware label smoothing, continent and country heads, run plan vs smoothed loss comparison

Jack Trask: Write metric code for error, accuracy at each distance threshold, GeoScore. Build the random baselines, most common country. Produce breakdowns by continent. Run final test set evaluation

Raghavan Rajkumar: Run geocell granularity and resolution studies, generate maps, build demo

7. Project Timeline

Our timeline for the project will follow the recommended timeline fairly closely. From September 22nd to September 24th, a reproducible preprocessing split will be made with the data, as well as a training pipeline. From the 25th of September to the 30th of September the first Deep Learning Model will be created, and an initial validation result will be found. Then, until October 4th, changes to the model will be made to better fit the data and give better validation results. Finally, by October 8th the final metrics will be found, and the project deliverables will be turned in. Finally on October 9th the final project will be presented to the class in a professional presentation.

References

[1] G. Astruc, N. Dufour, I. Siglidis, C. Aronssohn, N. Bouia, S. Fu, R. Loiseau, V. N. Nguyen, C. Raude, E. Vincent, L. Xu, H. Zhou, and L. Landrieu, "OpenStreetView-5M: The many roads to global visual geolocation," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2024.

Haas, L., Skreta, M., Alberti, S., & Finn, C. (2023). PIGEON: Predicting image geolocations. *arXiv preprint arXiv:2307.05845*.

OSV-5M Team, "OpenStreetView-5M dataset," Hugging Face, 2024. [Online]. Available: <https://huggingface.co/datasets/osv5m/osv5m> (license: CC-BY-SA 4.0)

Wi, J., Jang, Y., Lee, D. et al. Delving into Pre-training for Domain Transfer: A Broad Study of Pre-training for Domain Generalization and Domain Adaptation. *Int J Comput Vis* **134**, 50 (2026). <https://doi.org/10.1007/s11263-025-02590-5>